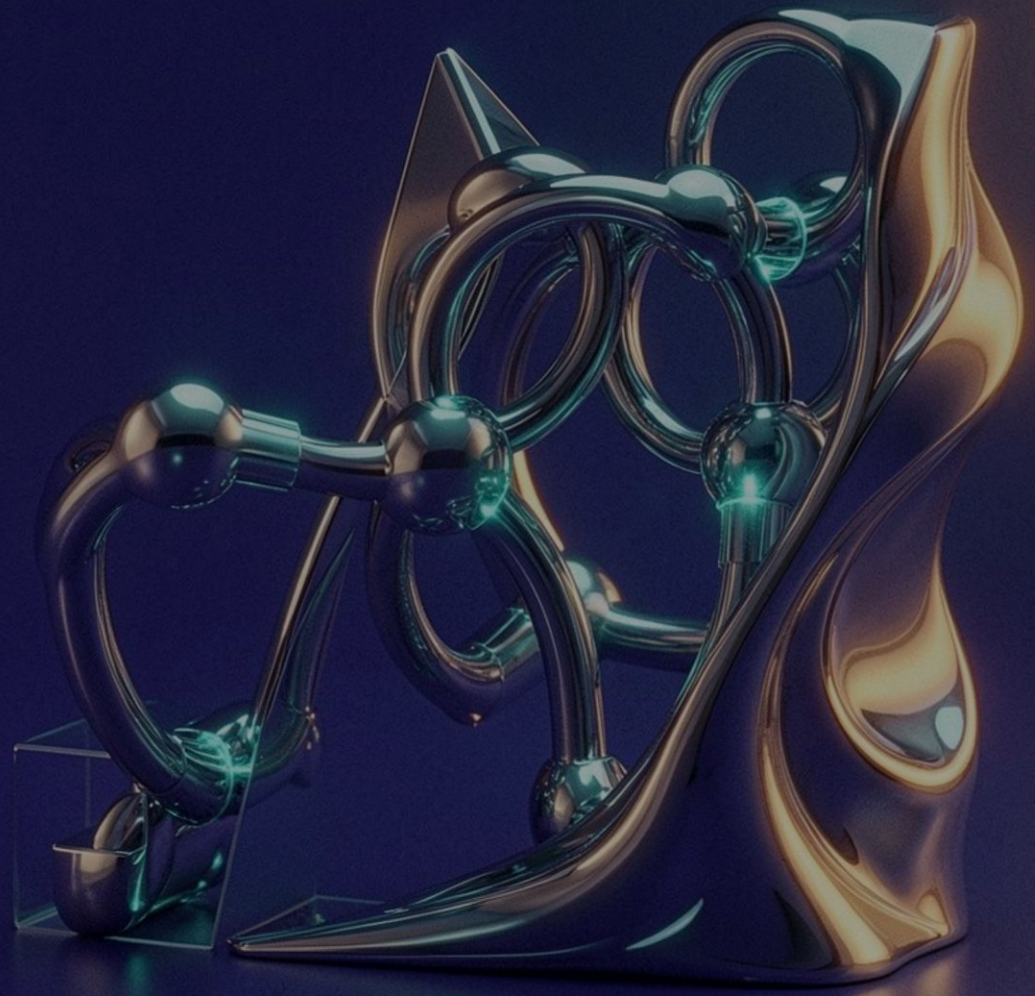
DESIGN & ANALYSIS OF ALGORITHMS

DIJKSTRA'S ALGORITHM

Single-Source Shortest Paths in Weighted Graphs



Siddhi Timkhede

PRN 25070521236 · Section 'D' · Semester III

Made By Gamma AI Tool

Made with GAMMA

DIJKSTRA'S ALGORITHM

What is Dijkstra's Algorithm?

Definition

Dijkstra's algorithm is a greedy graph-traversal algorithm used to find the shortest path from a single source vertex to every other reachable vertex in a weighted graph, provided that all edge weights are non-negative.

Fundamental concepts

- Graph: A collection of vertices (nodes) connected by edges.
- Weight: A numerical cost assigned to an edge, such as distance, time, or price.
- Shortest path: The path with the minimum total sum of edge weights between two vertices.
- Greedy approach: The algorithm repeatedly selects the unvisited vertex with the smallest tentative distance and finalises its shortest distance.
- Relaxation: The process of checking whether travelling through a particular vertex gives a shorter path to its neighbour.

DIJKSTRA'S ALGORITHM

Key Properties and Output

Key properties

- Single-source shortest path: Calculates the shortest distances from one source to all reachable vertices.
- Non-negative edge weights: The algorithm's correctness depends on all edge weights being zero or positive.
- Optimal substructure: A shortest path contains shortest paths between its intermediate vertices.
- Efficient path calculation: It updates tentative distances rather than enumerating every possible route.

Output: The algorithm produces the shortest distance from the source to each reachable vertex. Predecessor information can also be stored to reconstruct the actual routes.

THE GREEDY LOOP

How the Greedy Process Works

01

Initialisation

Assign distance 0 to the source vertex and infinity (∞) to all other vertices.

02

Selection

Choose the unvisited vertex with the smallest tentative distance.

03

Relaxation

Examine each unvisited neighbour and calculate the distance through the selected vertex.

04

Update

If the newly calculated distance is smaller than the existing distance, replace it and record the predecessor.

05

Finalisation

Mark the selected vertex as visited. Repeat the process until all reachable vertices have been processed.

THE GREEDY LOOP

Pseudocode and the Relaxation Rule

```
for each vertex v:
    dist[v] = ∞
    predecessor[v] = NIL

dist[source] = 0

while an unvisited reachable vertex exists:
    u = unvisited vertex with minimum dist[u]
    mark u as visited

    for each edge (u, v) with weight w:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            predecessor[v] = u
```

The relaxation rule:

$$dist[u] + w(u,v) < dist[v] \quad \Rightarrow \quad dist[v] = dist[u] + w(u,v)$$

The relaxation rule: If dist[u]+w(u,v)<dist[v], then dist[v]=dist[u]+w(u,v). Here, dist[u] is the current best-known distance to vertex u, and w(u,v) is the weight of the edge connecting u and v. The algorithm updates a distance only when it discovers a cheaper route.

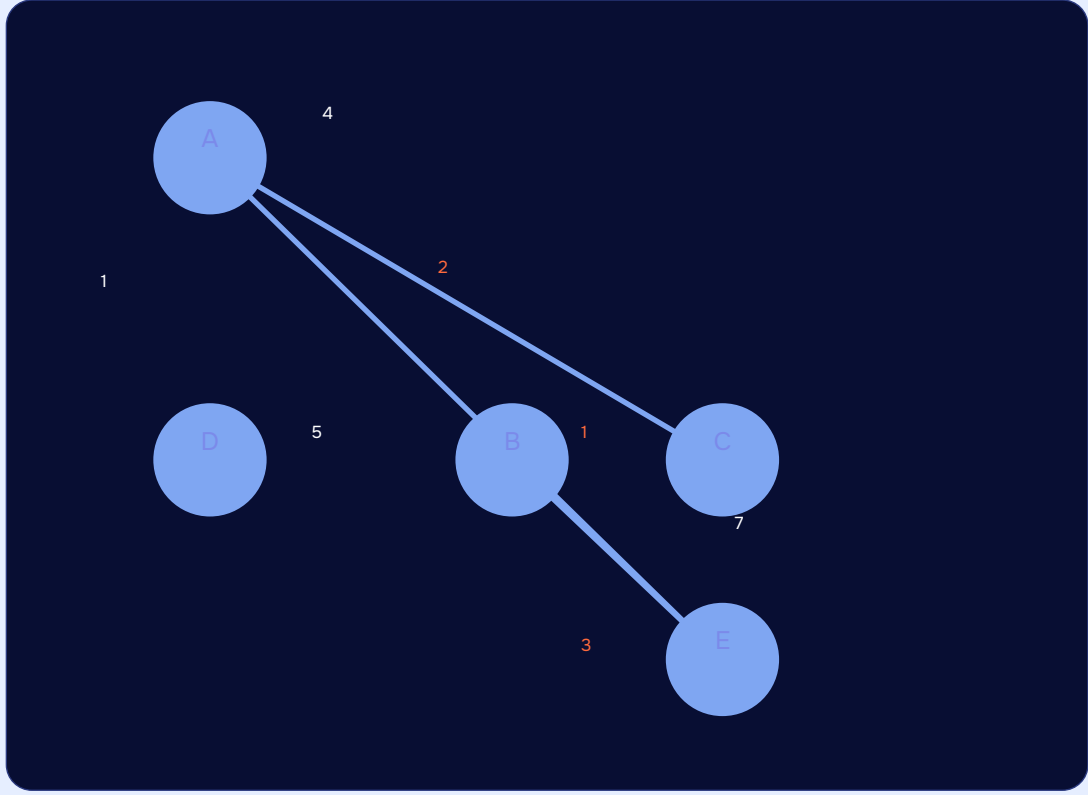
h GAMMA

WORKED EXAMPLE

Worked Example: Shortest Paths from A

Consider a weighted graph with five vertices, A, B, C, D and E. A is the source vertex. Use these weighted edges: A–B = 4, A–C = 1, C–B = 2, B–D = 1, C–D = 5, B–E = 7, D–E = 3.

The distance from A to C is 1. The distance from A to B through C is 1+2=3, which is shorter than the direct edge of weight 4. The distance from A to D through B is 1+2+1=4. The distance from A to E through D is 1+2+1+3=7. Therefore, the shortest distance from A to E is 7, following the route A → C → B → D → E.



Vertex	Shortest distance from A	Shortest route
A	0	A
B	3	A → C → B
C	1	A → C
D	4	A → C → B → D
E	7	A → C → B → D → E

Made with GAMMA

Step-by-Step Distance Updates

The table shows how tentative distances change as each vertex is processed. Infinity (∞) means that no route has been discovered yet.

Vertex processed	A	B	C	D	E
Initialisation	0	∞	∞	∞	∞
A	0	4	1	∞	∞
C	0	3	1	6	∞
B	0	3	1	4	10
D	0	3	1	4	7
E (complete)	0	3	1	4	7

1. Process A: Distance to B = 4; distance to C = 1. Since C has the smallest tentative distance, process C next.
2. Process C: Distance to B through C = 1+2=3, replacing 4. Distance to D through C = 1+5=6.
3. Process B: Distance to D through B = 3+1=4, replacing 6. Distance to E through B = 3+7=10.
4. Process D: Distance to E through D = 4+3=7, replacing 10.
5. Process E: E is finalised with distance 7. All reachable vertices now have their shortest distances from A.

The final distance array, in the order A, B, C, D, E, is [0, 3, 1, 4, 7].

Made with GAMMA

PRACTICAL VIEW

Efficiency and Complexity

Time and space complexity

Let V = number of vertices. Let E = number of edges.

Space complexity: O(V+E) for the graph represented using an adjacency list, plus distance, predecessor, and visited data. This is O(V+E) overall.

Implementation	Time complexity	Typical use
Array or adjacency matrix	$O(V^2)$	Small or dense graphs
Adjacency list + binary heap	$O((V+E) \log V)$	Common choice, especially for sparse graphs

Made with GAMMA

PRACTICAL VIEW

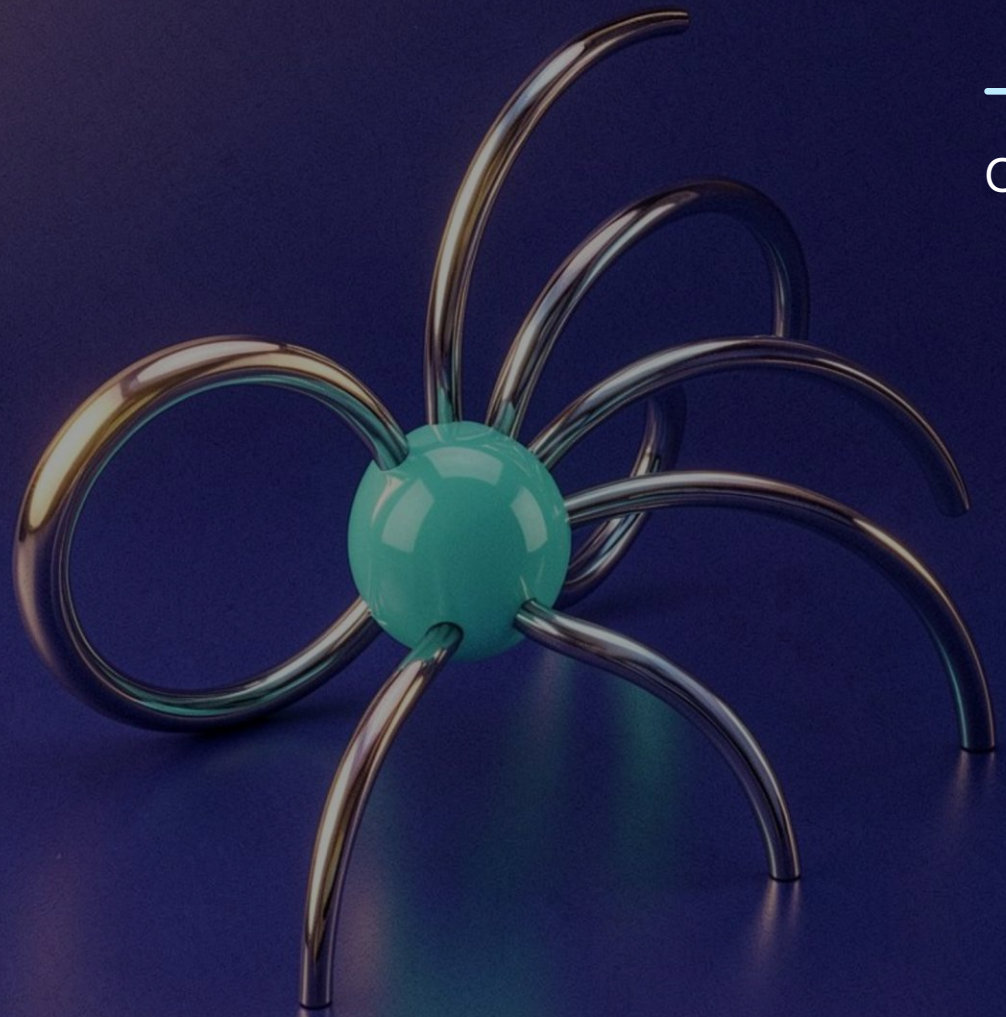
Applications, Limitations and Takeaway

Applications

- Navigation systems: Finding shortest routes between locations.
- Computer networks: Finding minimum-cost paths for packet routing.
- Robotics: Planning movement through weighted environments.
- Logistics: Identifying lower-cost delivery and transportation routes.
- Game development: Finding efficient paths for characters moving through game maps.

Limitations

- Negative edge weights: Dijkstra's algorithm is not generally correct when negative-weight edges are present. Bellman–Ford is a suitable alternative for single-source shortest paths when negative weights are allowed and there is no reachable negative cycle affecting the result.
- Single-source approach: It solves the problem from one source at a time. A different source generally requires another run.
- Implementation matters: The choice of graph representation and priority queue affects performance.
- Unreachable vertices: Vertices that cannot be reached from the source retain distance ∞.



THANK YOU

Questions & Discussion

Siddhi Timkhede · Semester III · Section D

Made with GAMMA